

OASIS INFOBYTE
DATA ANALYTICS INTERNSHIP
LEVEL 1 – TASK 3
DATA CLEANING AND PRELIMINARY ANALYSIS

Final Project Report

Submitted By

Madala Sai Sprisha

Internship

OASIS INFOBYTE – Data Analytics Internship

Domain

Data Analytics

Project Title

Level 1 – Task 3: Data Cleaning and Preliminary Analysis

Technologies Used

Python • Pandas • NumPy • Matplotlib • Seaborn • Jupyter Notebook

Modules Implemented

Data Quality Assessment • Missing Value Handling • Duplicate Removal • Data Standardisation • IQR
Outlier Detection • Final Validation • Cleaned Dataset Export

Submitted To

OASIS INFOBYTE

Year

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to OASIS INFOBYTE for providing me with the opportunity to complete Level 1 – Task 3, Data Cleaning and Preliminary Analysis. This project provided practical exposure to dataset inspection, missing-value treatment, duplicate analysis, data standardisation, outlier detection, validation, and cleaned-data export using Python.

I also thank the mentors and coordinators for providing the internship task requirements and learning resources. The structured workflow helped me understand how raw tabular data can be systematically prepared for reliable downstream analytics.

The implementation was completed in a Jupyter Notebook using the Titanic dataset. The project demonstrates a complete data-cleaning workflow from initial quality assessment to final validation and saving of the cleaned dataset.

The work strengthened my practical understanding of reproducible data preparation and demonstrated how careful quality checks can improve the reliability of later exploratory and predictive analysis.

ABSTRACT

This project focuses on data cleaning and preliminary analysis of the Titanic dataset using Python. The objective is to inspect the dataset, identify data-quality issues, handle missing values, remove duplicate records where applicable, standardize categorical and identifier fields, detect potential numerical outliers using the Interquartile Range (IQR) method, perform a final quality check, and save the cleaned dataset.

The input dataset contains 891 records and 12 columns. Initial inspection identified missing values in Age (177; 19.87%), Cabin (687; 77.10%), and Embarked (2; 0.22%). Age values were imputed using the median of 28.0, Embarked values were imputed using the mode 'S', and Cabin was removed because of its high missing proportion. No duplicate records were present. Potential outliers were identified in Age (66) and Fare (116) and retained because they may represent genuine observations.

After cleaning, the final dataset contains 891 records and 11 columns, with no missing values in retained fields and no duplicate rows. The cleaned dataset was exported as Titanic_Cleaned.csv and is ready for downstream analysis.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	Cover Page	1
	Abstract	2
	Acknowledgement	3
	Table of contents	4
Chapter	Introduction	
1.1	Project Overview	6
1.2	Problem Statement	6
1.3	Project Objectives	6
Chapter 2	Dataset & Tools	
2.1	Dataset Description	7
2.2	Tools & Technologies Used	7
2.3	Project Folder Structure	8
Chapter 3	Data Cleaning Implementation	
3.1	Dataset Loading	9
3.2	Data Quality Report	10
3.3	Missing Value Handling	11
3.4	Duplicate Removal	12
3.5	Data Standardization	13
3.6	Outlier Detection Using IQR	14
3.7	Final Data Quality Check	15
3.8	Cleaned Dataset Saved	16
Chapter 4	Results & Discussion	

4.1	Key Findings	17
4.2	Conclusion	18
4.3	Future Enhancements	18
4.4	References	18

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Data Cleaning and Preliminary Analysis project focuses on converting a raw Titanic passenger dataset into a consistent and analysis-ready dataset. The workflow begins with loading and inspecting the source data, followed by quality assessment, missing-value handling, duplicate checking, categorical standardisation, outlier detection, final validation, and export of the cleaned data.

The implementation uses Python and Pandas for data loading, inspection, transformation, validation, and CSV export. NumPy supports numerical operations, while Matplotlib and Seaborn provide visualization support. The complete implementation is documented in a Jupyter Notebook.

1.2 Problem Statement

Raw datasets frequently contain missing observations, inconsistent categorical formatting, duplicate records, and unusual numerical values. If these issues are not identified and handled appropriately, they can reduce the reliability of subsequent analysis. This project addresses these problems through a structured and reproducible data-cleaning workflow.

1.3 Project Objectives

1. Load the Titanic dataset into a Pandas DataFrame.
2. Generate a data-quality report covering structure, data types, missing values, duplicates, and numerical summaries.
3. Handle missing values using suitable imputation or column-removal decisions.
4. Identify and remove duplicate rows where present.
5. Standardize categorical values and identifier fields.
6. Detect potential outliers in numerical variables using the IQR method.
7. Perform a final data-quality validation.
8. Save the cleaned dataset as a CSV file for future analysis.

CHAPTER 2: DATASET & TOOLS

2.1 Dataset Description

The project uses the Titanic-Dataset.csv input file. The executed notebook reports an initial dataset shape of 891 rows and 12 columns. The columns are PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked.

2.2 Tools & Technologies Used

TOOL / TECHNOLOGY	PURPOSE
Python	Programming and data-cleaning workflow
Pandas	Data loading, inspection, cleaning and transformation
NumPy	Numerical operations and supporting analysis
Matplotlib	Visualization support
Seaborn	Statistical/data visualization support
Jupyter Notebook	Interactive implementation and documentation
CSV	Input and cleaned dataset storage

2.3 Project Folder Structure

The project is organized into separate folders for the dataset, notebook implementation, output file, report documentation, and screenshots. The evidence screenshots correspond to the eight major cleaning stages documented in Chapter 3.

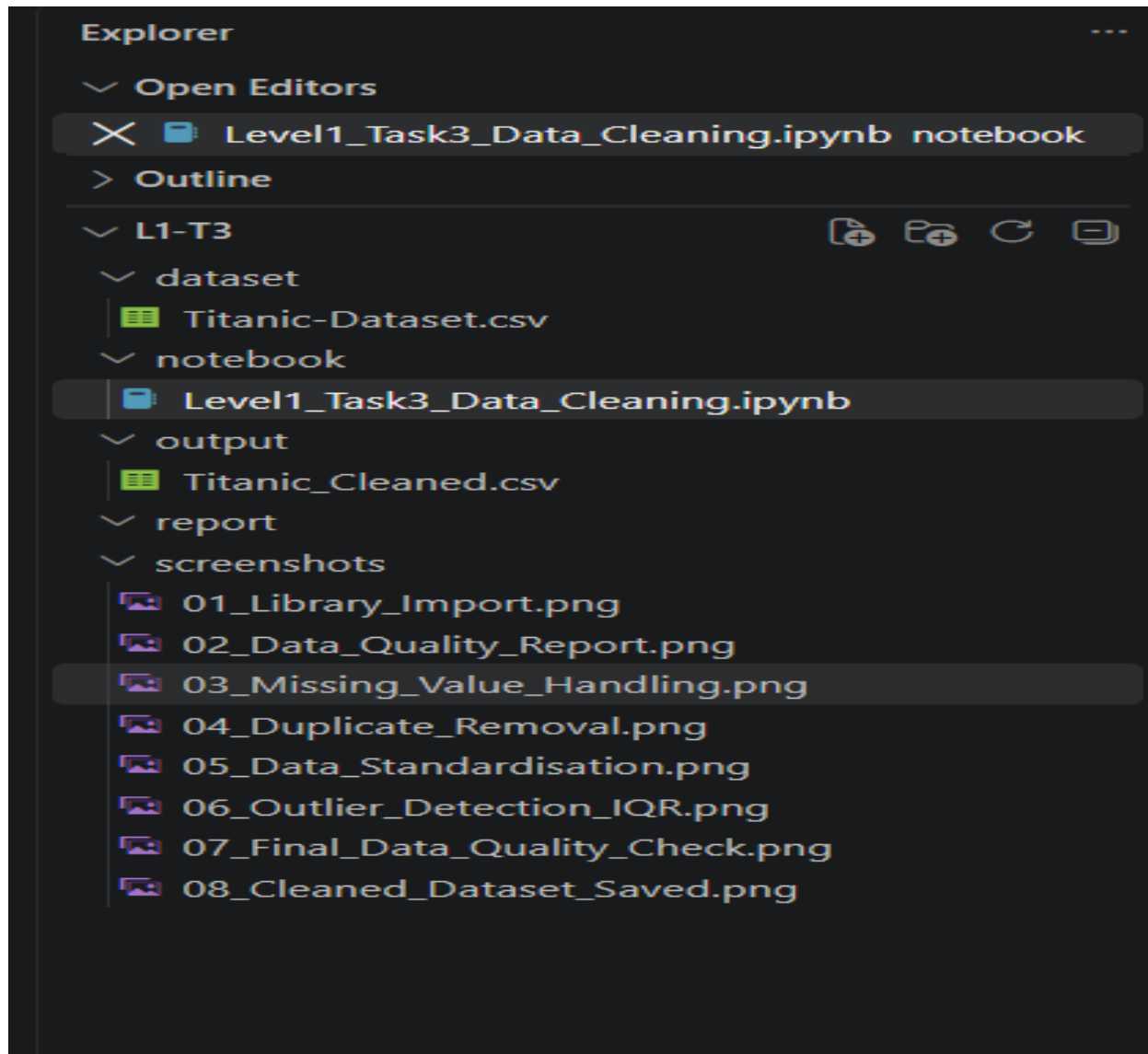


Figure 2.3: Project Folder Structure of the Data Cleaning and Preliminary Analysis Project

CHAPTER 3: DATA CLEANING IMPLEMENTATION

3.1 Dataset Loading

The required Python libraries were imported and the Titanic CSV file was loaded into a Pandas DataFrame. The notebook confirmed that the dataset file exists and reported an initial shape of 891 rows and 12 columns.

```
import os
file_path = "../dataset/ Titanic-Dataset.csv"
print("File exists:",os.path.exists(file_path))
print("Files in dataset folder:",os.listdir("../dataset"))

[5]
... File exists: False
Files in dataset folder: ['Titanic-Dataset.csv']

# Step 2: Data Quality Report

import pandas as pd

# Load dataset
file_path = "../dataset/Titanic-Dataset.csv"
df = pd.read_csv(file_path)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

# Dataset information
print("\n--- Dataset Information ---")
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

# Column names
print("\n--- Column Names ---")
print(df.columns.tolist())

# Data types

[7]
```

Figure 3.1: Dataset Loading Implementation

Interpretation:

The Titanic dataset was loaded successfully into a Pandas DataFrame. The initial dataset contains 891 rows and 12 columns, providing the complete passenger-level records required for the cleaning workflow.

3.2 Data Quality Report

The dataset was inspected using its shape, column names, data types, missing-value counts, duplicate counts, and descriptive statistics. This initial quality assessment was used to determine which fields required cleaning before further analysis.

```

# Missing values
print("\n--- Missing Values ---")
missing_values = df.isnull().sum()
display(missing_values)

# Duplicate rows
print("\n--- Duplicate Rows ---")
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:", duplicate_count)

# Numerical summary
print("\n--- Numerical Summary ---")
display(df.describe())

```

[7] Python

```

... Dataset loaded successfully!
Dataset shape: (891, 12)

--- Dataset Information ---
Number of rows: 891
Number of columns: 12

--- Column Names ---
['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'Si]

--- Data Types ---

```

Column	Data Type
PassengerId	int64
Survived	int64
Pclass	int64
Name	object
Sex	object
Age	float64
SibSp	int64

Figure 3.2: Initial Data Quality Report

Interpretation: The initial quality assessment identified missing values in Age, Cabin, and Embarked. Age contains 177 missing values (19.87%), Cabin contains 687 (77.10%), and Embarked contains 2 (0.22%). No duplicate rows were identified.

3.3 Missing Value Handling

Missing values were handled using decisions documented in the notebook. Age was treated with median imputation, Embarked with mode imputation, and Cabin was removed because its missing proportion was very high.

```
df['Embarked'] = df['Embarked'].fillna(embarked_mode)

# Cabin: Drop because 77.10% of values are missing
df = df.drop(columns=['Cabin'])

print("Missing data handling completed successfully!")
print("Age median used:", age_median)
print("Embarked mode used:", embarked_mode)
print("Cabin column removed due to high missing percentage.")

print("\nMissing values after handling:")
display(df.isnull().sum())
```

[10]

```
... Missing data handling completed successfully!
Age median used: 28.0
Embarked mode used: S
Cabin column removed due to high missing percentage.

Missing values after handling:

... PassengerId    0
Survived          0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Embarked         0
dtype: int64
```

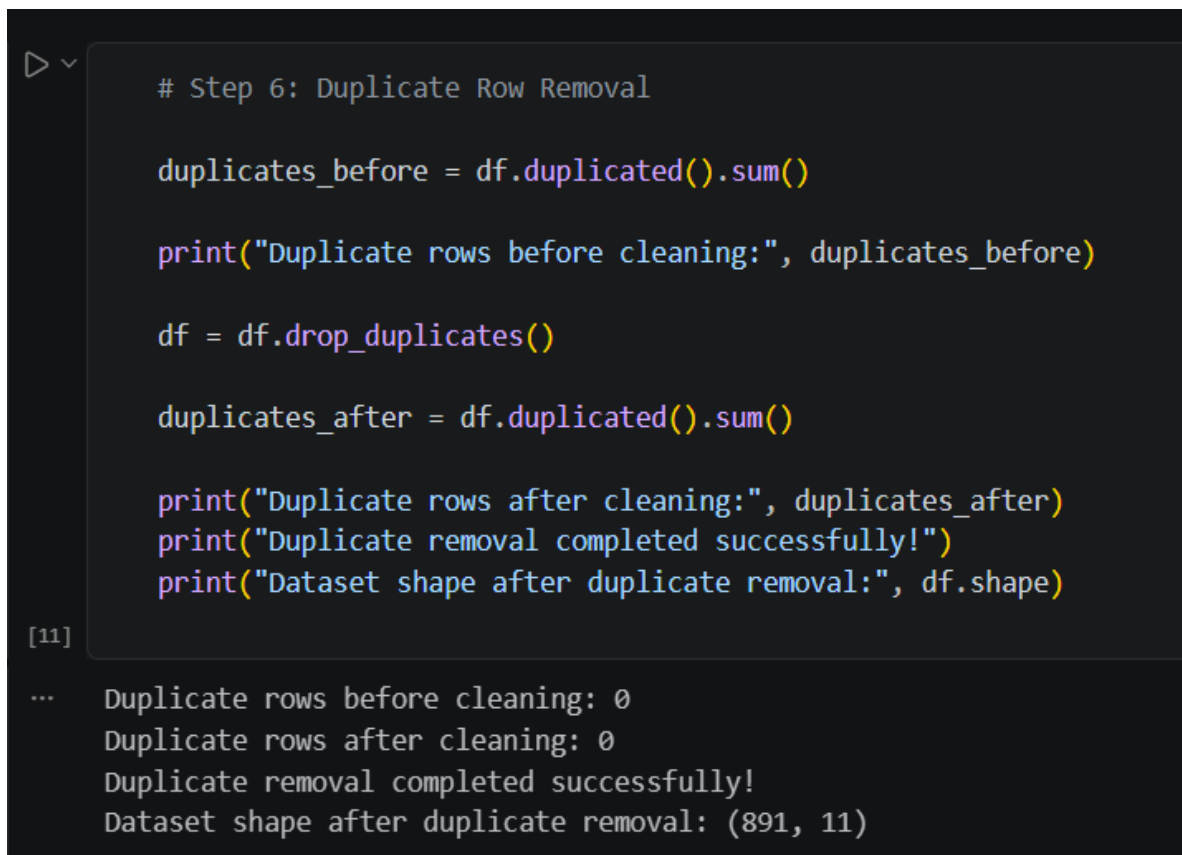
Figure 3.3: Missing Value Handling

Interpretation:

Age was imputed with the median value of 28.0 because it is numerical and the median is less sensitive to extreme values. Embarked was imputed with the mode 'S' because only two observations were missing. Cabin was removed because 77.10% of its values were missing. The retained columns contained no missing values after treatment.

3.4 Duplicate Removal

Duplicate rows were checked before and after the cleaning operation. The notebook applied `drop_duplicates()` so that the workflow remains reproducible even when duplicate rows are encountered in other versions of the input data.



```
# Step 6: Duplicate Row Removal

duplicates_before = df.duplicated().sum()

print("Duplicate rows before cleaning:", duplicates_before)

df = df.drop_duplicates()

duplicates_after = df.duplicated().sum()

print("Duplicate rows after cleaning:", duplicates_after)
print("Duplicate removal completed successfully!")
print("Dataset shape after duplicate removal:", df.shape)
```

[11]

```
... Duplicate rows before cleaning: 0
Duplicate rows after cleaning: 0
Duplicate removal completed successfully!
Dataset shape after duplicate removal: (891, 11)
```

Figure 3.4: Duplicate Record Check and Removal

Interpretation:

No duplicate records were present before cleaning, and the duplicate count remained zero after the removal step. The operation therefore preserved all 891 passenger records while confirming that duplicate rows were not a data-quality issue in this dataset.

3.5 Data Standardisation

Categorical values and identifier formatting were standardised. The Sex column was converted to a consistent lowercase representation, Embarked was standardized to uppercase, and PassengerId was converted to string because it is an identifier rather than a numerical measurement.

```

# Standardise Sex values
df['Sex'] = df['Sex'].astype(str).str.strip().str.lower()

# Standardise Embarked values
df['Embarked'] = df['Embarked'].astype(str).str.strip().str.upper()

# Convert PassengerId to string because it is an identifier
df['PassengerId'] = df['PassengerId'].astype(str)

print("Data standardisation completed successfully!")

print("\nUnique Sex values:")
print(df['Sex'].unique())

print("\nUnique Embarked values:")
print(df['Embarked'].unique())

print("\nUpdated data types:")
display(df.dtypes)

```

[12]

```

... Data standardisation completed successfully!

Unique Sex values:
['male' 'female']

Unique Embarked values:
['S' 'C' 'Q']

Updated data types:
... PassengerId      object
   Survived         int64
   Pclass          int64
   Name            object

```

Figure 3.5: Data Standardization

Interpretation:

The Sex values were standardized to lowercase and the Embarked values to uppercase, producing consistent categorical representations. PassengerId was converted to string because it is an identifier rather than a numerical measurement.

3.6 Outlier Detection Using IQR

Potential outliers were detected in the numerical variables Age and Fare using the Interquartile Range (IQR) rule. The IQR is calculated as $Q3 - Q1$, and observations below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are treated as potential outliers.

```
(df[column] > upper_bound)
]

outlier_summary.append({
    'Column': column,
    'Q1': Q1,
    'Q3': Q3,
    'IQR': IQR,
    'Lower Bound': lower_bound,
    'Upper Bound': upper_bound,
    'Outlier Count': len(outliers)
})

outlier_report = pd.DataFrame(outlier_summary)

print("Outlier detection completed successfully!")
display(outlier_report)
```

[14] Python

... Outlier detection completed successfully!

...

	Column	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count
0	Age	22.0000	35.0	13.0000	2.500	54.5000	66
1	Fare	7.9104	31.0	23.0896	-26.724	65.6344	116

```
### Outlier Treatment Decision

The IQR method identified 66 potential outliers in Age and 116 potential outliers in Fare. These observations were retained because the extreme values may represent genuine passenger characteristics, such as older passengers or passengers who paid higher fares. Removing them could result in loss of meaningful information. Therefore, the identified outliers were documented but not removed from the cleaned dataset.
```

markdown

Figure 3.6: Outlier Detection Using the IQR Method

Interpretation:

The IQR method identified 66 potential outliers in Age and 116 in Fare. These observations were retained rather than removed because they may represent genuine passenger characteristics or legitimate high-fare observations. The analysis therefore documents potential outliers without unnecessarily discarding information.

3.7 Final Data Quality Check

A final validation was performed after all cleaning operations. The final shape, missing values, duplicate count, data types, and numerical summary were reviewed before exporting the cleaned data.

```
# Step 9: Final Data Quality Check

print("--- Final Data Quality Check ---")

print("Final dataset shape:", df.shape)

print("\nMissing values after cleaning:")
display(df.isnull().sum())

print("\nDuplicate rows after cleaning:")
print(df.duplicated().sum())

print("\nFinal data types:")
display(df.dtypes)

print("\nFinal numerical summary:")
display(df.describe())
```

[15]

```
... --- Final Data Quality Check ---
Final dataset shape: (891, 11)

Missing values after cleaning:

... PassengerId    0
Survived          0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Embarked         0
```

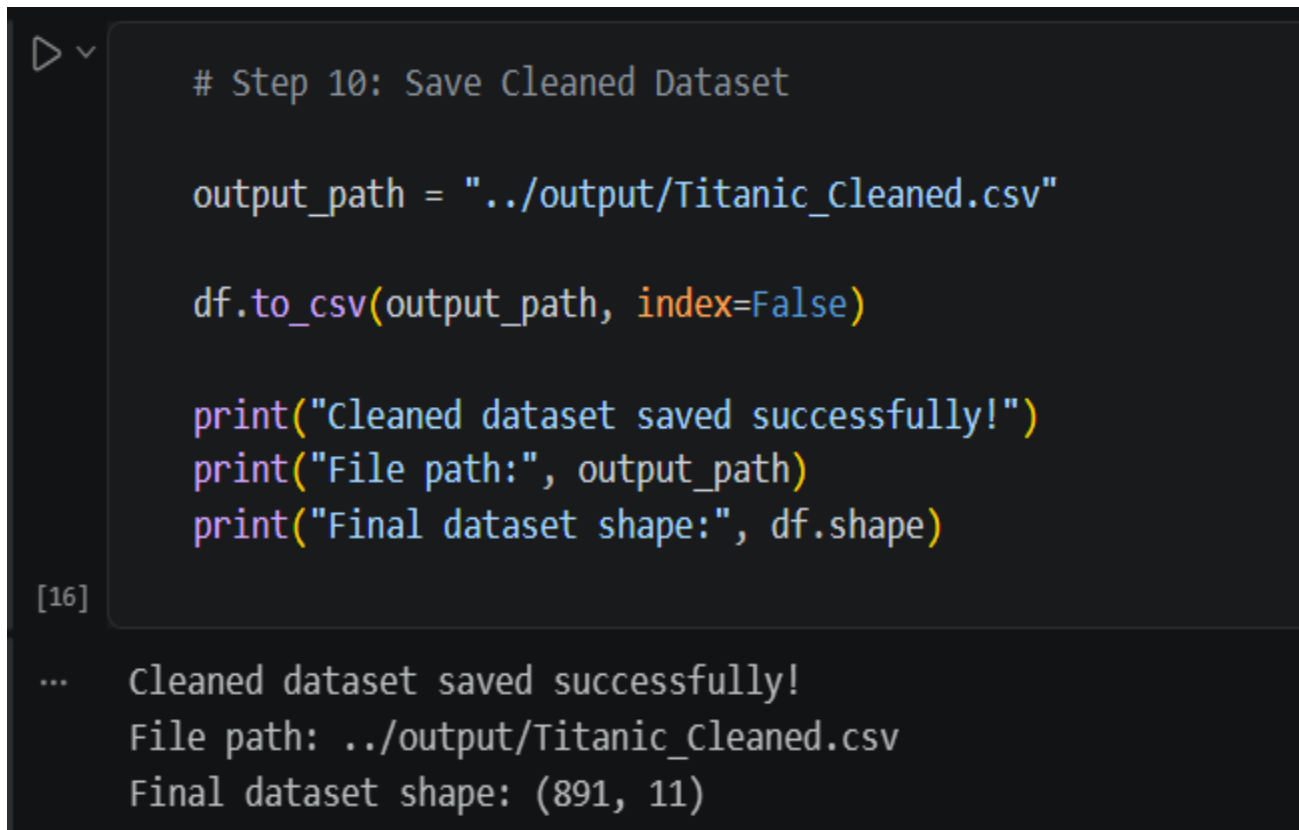
Figure 3.7: Final Data Quality Validation

Interpretation:

The final validation confirms a cleaned dataset shape of 891 rows and 11 columns. All retained columns contain zero missing values and there are zero duplicate rows. The final data types and numerical summary were also reviewed before export.

3.8 Cleaned Dataset Saved

The cleaned DataFrame was exported to the output folder as a CSV file. This creates a reusable, analysis-ready dataset for subsequent exploratory or predictive work.



```
# Step 10: Save Cleaned Dataset

output_path = "../output/Titanic_Cleaned.csv"

df.to_csv(output_path, index=False)

print("Cleaned dataset saved successfully!")
print("File path:", output_path)
print("Final dataset shape:", df.shape)
```

[16]

```
... Cleaned dataset saved successfully!
File path: ../output/Titanic_Cleaned.csv
Final dataset shape: (891, 11)
```

Figure 3.8: Cleaned Dataset Export

Interpretation:

The cleaned dataset was successfully exported as Titanic_Cleaned.csv. The final output contains 891 records and 11 columns and is ready for downstream analysis.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Key Findings

METRIC	RESULT
Initial dataset	891 rows $\times$ 12 columns
Initial missing values	Age: 177 (19.87%); Cabin: 687 (77.10%); Embarked: 2 (0.22%)
Duplicate rows	0 identified
Missing-value treatment	Age median = 28.0; Embarked mode = S; Cabin removed
Standardization	Sex $\rightarrow$ lowercase; Embarked $\rightarrow$ uppercase; PassengerId $\rightarrow$ string
Potential outliers	Age: 66; Fare: 116
Final dataset	891 rows $\times$ 11 columns
Final missing values	0 in all retained columns
Final duplicate rows	0
Output file	Titanic_Cleaned.csv

The results show that the cleaning workflow resolved all missing values in the retained fields, confirmed that duplicate rows were not present, standardized the selected categorical and identifier fields, and documented potential numerical outliers without automatically deleting potentially meaningful observations.

4.2 Conclusion

The Level 1 – Task 3 Data Cleaning and Preliminary Analysis project was successfully completed using Python and Pandas. The workflow systematically assessed the Titanic dataset, treated missing observations, confirmed the absence of duplicate rows, standardized categorical and identifier fields, detected potential outliers using the IQR method, and validated the final dataset.

The final cleaned dataset contains 891 records and 11 columns, with no missing values in retained fields and no duplicate rows. Potential outliers were documented rather than automatically removed, preserving observations that may carry legitimate information. The resulting Titanic_Cleaned.csv file provides a reliable foundation for subsequent exploratory or predictive analysis.

4.3 Future Enhancements

1. Perform exploratory data analysis using distributions and relationship visualisations.
2. Investigate whether additional feature engineering can improve downstream modelling.
3. Evaluate alternative outlier-treatment strategies and compare their effect on analysis.
4. Encode categorical variables and prepare a modelling-ready dataset when required.
5. Build predictive models for survival analysis using the cleaned dataset.

4.4 References

Project-provided Titanic-Dataset.csv and Level 1 – Task 3 implementation notebook.

Pandas library functionality used for data loading, missing-value handling, duplicate removal, transformation, and CSV export.

Python data-analysis environment and Jupyter Notebook used for interactive implementation.